

Handoff: Semantic Authority & Performance Mapper (v2)

1. Visión general del sistema

El sistema procesa la estructura completa de un sitio web para generar un **mapa vectorial semántico 2D** donde:

- La **posición** de cada URL refleja su significado semántico (embeddings).
- La **masa** (tamaño del punto) depende de su autoridad estructural (PageRank interno) y su rendimiento en clics (GSC).
- El **color** indica su rol en la arquitectura semántica: núcleo, periferia, outlier o par de canibalización.

Salidas principales:

- **Scatter UMAP** interactivo con clusters temáticos HDBSCAN (vista de análisis técnico).
- **Mapa de anillos concéntricos** con clasificación Core / Focus / Expansion / Peripheral (vista de presentación a cliente).
- Ranking de URLs que desvían el centroide semántico del sitio.
- Detección de pares de canibalización por similitud coseno.
- Análisis de brechas: vector desde el centroide hacia un topic objetivo.
- Informe CSV exportable con todas las métricas por URL.

2. Stack tecnológico

capa	herramienta	motivo
Extracción HTML	trafilatura + Playwright (fallback)	trafilatura cubre CMS clásicos; Playwright cubre SPAs y JS rendering
	text-embedding-3-large	

Embeddings	(OpenAI) o <code>voyage-large-2-instruct</code>	Voyage superior en tareas retrieval; OpenAI más económico
Procesamiento	pandas, numpy, scikit-learn	estándar
Reducción dimensional	PCA (50 dims) → UMAP	PCA primero evita colapso de UMAP en corpus grandes
Grafos	NetworkX	cálculo de PageRank interno
Clustering	HDBSCAN	no requiere fijar k; robusto a ruido
Visualización	Plotly (interactivo)	hover, zoom, filtros por cluster
UI	Streamlit	bajo coste de desarrollo

3. Inputs requeridos

crawl.csv — exportación Screaming Frog

columna	tipo	notas
<code>url</code>	string	URL canónica
<code>status_code</code>	int	procesar solo 200
<code>indexable</code>	bool	excluir noindex
<code>inlinks</code>	int	enlaces internos entrantes
<code>html</code>	string	exportar con "store HTML" activado

gsc.csv — exportación Google Search Console

Periodo recomendado: 90 días.

columna	tipo	notas
<code>url</code>	string	debe coincidir con crawl.csv tras normalización
<code>clicks</code>	int	clics totales

impressions	int	impresiones totales
position	float	posición media

Normalización obligatoria antes del join: lowercase, eliminar trailing slash, eliminar parámetros UTM (`utm_source` , `utm_medium` , `utm_campaign` , `fbclid` , `gclid`). Sin este paso, el merge pierde entre un 20% y un 40% de URLs en sitios con tracking activo.

URLs sin datos en GSC: Es normal que el 30–60% de URLs indexables no tengan clics registrados (páginas nuevas, facets, paginación). No se excluyen del mapa. Reciben `clicks_norm = 0` y su peso queda determinado únicamente por PageRank (ver sección 4.B).

4. Algoritmos y modelado matemático

A. Extracción de contenido principal

trafilatura falla silenciosamente con SPAs y contenido renderizado con JavaScript. El pipeline usa extracción en dos fases:

```
def extract_main_content(html: str, url: str) -> str | None:
    """
    Fase 1: extracción estática con trafilatura.
    Fase 2: si el resultado es None o < 150 tokens, renderizado
            con Playwright y segundo intento.
    Umbral mínimo: 150 tokens. Por debajo se marca como
    'low_content' y se excluye del mapa semántico.
    """
    text = trafilatura.extract(html, include_comments=False,
                               include_tables=False,
                               favor_precision=True)
    if text and len(text.split()) >= 150:
        return text

    # Fallback: Playwright
    text = playwright_extract(url)
    if text and len(text.split()) >= 150:
        return text

    return None # Marcada como low_content, excluida del pipeline
```

B. Vectorización: selección de chunk representativo

Problema del max-pooling dimensional: tomar el valor máximo por cada dimensión de forma independiente produce un vector que no corresponde a ningún chunk real del documento. Es geoméricamente incoherente y no representa ninguna entidad semántica concreta.

Solución: chunk representativo por proximidad al centroide del documento.

```
def get_representative_chunk(chunks: list[str], model) -> np.ndarray:
    """
    1. Genera embeddings para todos los chunks del documento.
    2. Calcula el centroide del documento (media aritmética).
    3. Selecciona el chunk cuyo embedding tiene mayor similitud
       coseno con el centroide.
    4. Ese chunk es el vector de página: representa el tema
       dominante sin distorsión geométrica.
    """
    embeddings = np.array(model.embed(chunks))
    centroid = embeddings.mean(axis=0)
    similarities = cosine_similarity([centroid], embeddings)[0]
    best_idx = similarities.argmax()
    return embeddings[best_idx]
```

Parámetros de chunking:

- Tamaño: 500 tokens
- Overlap: 10% (50 tokens)
- Separador preferido: párrafo (\n\n), con fallback a oración

C. Centroide pesado del sitio

El centroide semántico del sitio c_{site} es la media ponderada de todos los vectores de página, donde el peso w_i combina PageRank normalizado y clics normalizados:

$$w_i = \alpha \cdot \text{Norm}(\text{PR}_i) + \beta \cdot \text{Norm}(\text{Ck}_i)$$

$$\alpha + \beta = 1 \quad (\text{hiperparámetros configurables, default: } \alpha=0.6, \beta=0.4)$$

$$\text{Norm}(x) = (x - x_{\min}) / (x_{\max} - x_{\min}) \quad [\text{Min-Max, rango } 0-1]$$

$$c_{site} = \sum(w_i \cdot V_i) / \sum(w_i)$$

Caso URLs sin clics ($ck_i = 0$):

- $\text{Norm}(\text{Ck}_i) = 0$

- Su peso queda en $w_i = \alpha \cdot \text{Norm}(\text{PR}_i)$
- Participan en el cálculo del centroide con menor influencia
- No se excluyen: un PageRank alto sin clics puede indicar contenido estructuralmente importante no descubierto aún

Caso sitio sin datos GSC en absoluto:

- Modo degradado: $\beta = 0, \alpha = 1$
- El sistema emite warning explícito en log

D. Detección de outliers semánticos (IQR)

$d_i = ||V_i - C_{\text{site}}||_2$ (distancia euclídea de cada página al centroide)

$Q1, Q3 = \text{percentiles } 25 \text{ y } 75 \text{ de } \{d_i\}$

$IQR = Q3 - Q1$

Outlier si: $d_i > Q3 + 1.5 \cdot IQR$

Periférico si: $Q3 < d_i \leq Q3 + 1.5 \cdot IQR$ [zona gris, no marcada como outlier]

Core si: $d_i \leq Q3$

E. Detección de canibalización semántica

Dos URLs canibalizan si compiten por el mismo espacio semántico y ninguna tiene autoridad estructural dominante clara.

`CANNIBAL_THRESHOLD = 0.92 # similitud coseno`

```
def detect_cannibalization(vectors: dict[str, np.ndarray],
                           weights: dict[str, float]) -> list[tuple]:
    pairs = []
    urls = list(vectors.keys())
    for i in range(len(urls)):
        for j in range(i + 1, len(urls)):
            sim = cosine_similarity([vectors[urls[i]]],
                                   [vectors[urls[j]]])[0][0]
            if sim >= CANNIBAL_THRESHOLD:
                # Identificar cuál de las dos tiene más autoridad
                dominant = urls[i] if weights[urls[i]] >= weights[urls[j]] else u
                weak = urls[j] if dominant == urls[i] else urls[i]
                pairs.append({
                    "url_dominant": dominant,
                    "url_weak": weak,
```

```

        "cosine_similarity": round(sim, 4),
        "recommendation": "consolidar en dominant o diferenciar topic
    })
    return sorted(pairs, key=lambda x: x["cosine_similarity"], reverse=True)

```

El umbral `0.92` es conservador. Para sitios de ecommerce con fichas de producto muy similares puede bajarse a `0.88`. Para sitios editoriales con artículos temáticamente cercanos pero diferenciados, puede subirse a `0.95`.

5. Pipeline completo

```

class SemanticEngine:
    def __init__(self, alpha: float = 0.6, beta: float = 0.4,
                 cannibal_threshold: float = 0.92):
        assert abs(alpha + beta - 1.0) < 1e-6, "α + β debe ser igual a 1"
        self.alpha = alpha
        self.beta = beta
        self.cannibal_threshold = cannibal_threshold

    def process_site(self, crawl_path: str, gsc_path: str) -> dict:

        # 1. Carga y filtrado
        crawl = pd.read_csv(crawl_path)
        crawl = crawl[(crawl.status_code == 200) & (crawl.indexable == True)]
        crawl = crawl[crawl.content_type.str.contains("text/html", na=False)]

        gsc = pd.read_csv(gsc_path)

        # 2. Normalización de URLs y join
        crawl["url_norm"] = crawl["url"].apply(normalize_url)
        gsc["url_norm"] = gsc["url"].apply(normalize_url)
        df = crawl.merge(gsc[["url_norm", "clicks", "impressions", "position"]],
                        on="url_norm", how="left")
        df["clicks"] = df["clicks"].fillna(0)

        # 3. Extracción de contenido y vectorización
        df["content"] = df.apply(
            lambda r: extract_main_content(r["html"], r["url"]), axis=1)
        df = df[df["content"].notna()] # Excluir low_content

        df["chunks"] = df["content"].apply(chunk_text)
        df["vector"] = df["chunks"].apply(
            lambda c: get_representative_chunk(c, embedding_model))

```

```

# 4. PageRank interno
G = build_link_graph(crawl) # Nodos: URLs, Aristas: enlaces internos
pr = nx.pagerank(G, weight="link_count", alpha=0.85)
df["pagerank"] = df["url"].map(pr).fillna(0)

# 5. Pesos combinados
df["pr_norm"] = minmax_normalize(df["pagerank"])
df["clicks_norm"] = minmax_normalize(df["clicks"])
df["weight"] = (self.alpha * df["pr_norm"] +
                self.beta * df["clicks_norm"])

# 6. Centroide pesado
vectors = np.vstack(df["vector"].values)
weights = df["weight"].values
centroid = np.average(vectors, axis=0, weights=weights)

# 7. Distancias y detección de outliers
df["distance_to_centroid"] = np.linalg.norm(
    vectors - centroid, axis=1)
df["semantic_role"] = classify_by_iqr(df["distance_to_centroid"])

# 8. Reducción dimensional: PCA → UMAP
# PCA a 50 dims primero: reduce coste computacional de UMAP
# y evita el colapso de la proyección en corpus > 5.000 URLs
n_components_pca = min(50, vectors.shape[0] - 1, vectors.shape[1])
pca = PCA(n_components=n_components_pca)
vectors_pca = pca.fit_transform(vectors)

n_neighbors = min(15, len(df) - 1) # Adaptativo al tamaño del corpus
umap_model = UMAP(n_neighbors=n_neighbors,
                  min_dist=0.1,
                  metric="cosine",
                  random_state=42)
coords_2d = umap_model.fit_transform(vectors_pca)
df[["x", "y"]] = coords_2d

# 9. Clustering temático post-UMAP
clusterer = hdbscan.HDBSCAN(min_cluster_size=max(5, len(df) // 50),
                             metric="euclidean")
df["cluster"] = clusterer.fit_predict(coords_2d)
# cluster == -1 → ruido / no asignado

# 10. Canibalización semántica
vectors_dict = dict(zip(df["url"], df["vector"]))
weights_dict = dict(zip(df["url"], df["weight"]))
cannibalization_pairs = detect_cannibalization(

```

```

        vectors_dict, weights_dict, self.cannibal_threshold)

    return {
        "dataframe": df,
        "centroid": centroid,
        "cannibalization": cannibalization_pairs,
    }

```

6. Reducción dimensional: configuración adaptativa

UMAP con parámetros fijos es una trampa. `n_neighbors=15` produce clusters artificiales en corpus pequeños y es computacionalmente inviable en corpus grandes sin reducción previa.

tamaño del corpus	estrategia
< 200 URLs	UMAP directo, <code>n_neighbors = max(5, n//10)</code>
200–5.000 URLs	PCA a 50 dims → UMAP, <code>n_neighbors=15</code>
> 5.000 URLs	PCA a 50 dims → UMAP con <code>low_memory=True</code> , <code>n_neighbors=30</code>

```

def get_umap_config(n_urls: int) -> dict:
    if n_urls < 200:
        return {"n_neighbors": max(5, n_urls // 10), "low_memory": False}
    elif n_urls < 5000:
        return {"n_neighbors": 15, "low_memory": False}
    else:
        return {"n_neighbors": 30, "low_memory": True}

```

7. Visualización interactiva

El sistema ofrece dos modos de visualización complementarios, seleccionables desde la UI. Ninguno reemplaza al otro: tienen audiencias y propósitos distintos.

modo	audiencia	propósito
Scatter UMAP (análisis)	SEO técnico / analista	Detectar clusters temáticos reales, canibalización, estructura

Anillos concéntricos (cliente)	Cliente / dirección	Comunicar salud semántica del sitio de forma inmediata
--------------------------------	---------------------	--

Modo A: scatter UMAP (análisis técnico)

capa	codificación visual
Posición (x, y)	Proyección UMAP sobre embeddings
Tamaño del punto	Proporcional a <code>weight</code> (PageRank + clics)
Color del punto	Cluster HDBSCAN (paleta categórica)
Borde del punto	Rojo si <code>semantic_role == "outlier"</code>
Líneas	Semitransparentes entre puntos del mismo cluster
Marcador especial	Diamante negro = centroide del sitio

Modo B: anillos concéntricos (presentación a cliente)

Inspirado en la visualización de Go Fish Digital, adaptado con pesos propios. Las distancias al centroide se calculan en el espacio de alta dimensión (no en UMAP 2D), lo que garantiza que los anillos reflejen distancia semántica real y no artefactos de la proyección.

Clasificación por cuartiles dinámicos (IQR sobre no-outliers):

Los anillos no tienen radios fijos: se adaptan automáticamente a la dispersión de cada sitio. Esto es crucial: un sitio muy especializado tendrá anillos más estrechos que uno generalista, sin que el analista tenga que calibrar umbrales manualmente.

```
def classify_rings(df: pd.DataFrame) -> pd.DataFrame:
    """
    Outliers: IQR clásico (Q3 + 1.5·IQR) sobre todas las distancias.
    Para las URLs no-outlier, se dividen en tres anillos por cuartiles:
    - Core:      Q0-Q1 (25% más cercanas al centroide)
    - Focus:     Q1-Q3 (50% centrales)
    - Expansion: Q3-max_non_outlier (25% más alejadas, sin ser outlier)
    """
    d = df["distance_to_centroid"]
    Q1, Q3 = d.quantile(0.25), d.quantile(0.75)
    IQR = Q3 - Q1
```

```

outlier_threshold = Q3 + 1.5 * IQR

conditions = [
    d > outlier_threshold,
    d > Q3,
    d > Q1,
    d <= Q1,
]

labels = ["Peripheral", "Expansion", "Focus", "Core"]
df["ring"] = np.select(conditions, labels)

# Métricas de resumen del sitio
ring_counts = df["ring"].value_counts()
focus_score = round(
    100 * (1 - df[df["ring"] == "Peripheral"]["weight"].sum() /
           df["weight"].sum()), 1)

return df, {
    "focus_score": focus_score,          # 0-100, mayor = más enfocado
    "semantic_radius": round(d.median(), 4),
    "drift_score": round(d[d > outlier_threshold].mean(), 4),
    "ring_counts": ring_counts.to_dict(),
}

```

Codificación visual de los anillos:

anillo	color	distancia	significado
Core	verde	Q0–Q1	25% de páginas más alineadas con el topic principal
Focus	azul	Q1–Q3	50% de páginas de soporte temático
Expansion	naranja	Q3–outlier	25% de páginas en expansión temática legítima
Peripheral	rojo	> outlier	Páginas que distorsionan el foco semántico del sitio

Métricas del dashboard de anillos:

Focus Score: 88.1 (0-100, mayor = sitio más enfocado)
 Semantic Radius: 87.5 (distancia mediana al centroide, en espacio original)
 Drift Score: 0.6 (distancia media de páginas Peripheral)
 Total páginas: 318

Diferencia clave respecto a Go Fish Digital: sus anillos usan centroide sin ponderar (media simple de todos los embeddings). El nuestro usa centroide pesado por PageRank

y clics, de modo que páginas con alta autoridad y tráfico anclan el centro semántico. En sitios con mucho contenido de baja calidad o huérfano, esto produce un centroide más representativo del topic real del negocio y no del promedio bruto de todo lo publicado.

Renderizado con Plotly:

```
def build_ring_map(df: pd.DataFrame, site_metrics: dict) -> go.Figure:
    """
    Coordenadas del scatter: usar las mismas coords_2d del UMAP,
    pero el radio visual de cada anillo se calcula proporcionalmente
    a los cuartiles de distancia real (no a las coords 2D).
    El centro visual es siempre el centroide proyectado en 2D.
    """
    color_map = {
        "Core": "#4CAF50",
        "Focus": "#2196F3",
        "Expansion": "#FF9800",
        "Peripheral": "#F44336",
    }

    fig = go.Figure()

    # Anillos de fondo (círculos SVG con relleno semitransparente)
    for ring, radius_quantile in [("Peripheral", 1.0),
                                   ("Expansion", 0.75),
                                   ("Focus", 0.50),
                                   ("Core", 0.25)]:
        fig.add_shape(type="circle",
                       xref="x", yref="y",
                       x0=-radius_quantile, y0=-radius_quantile,
                       x1=radius_quantile, y1=radius_quantile,
                       line=dict(color=color_map[ring], dash="dash", width=1.5),
                       fillcolor=color_map[ring],
                       opacity=0.06)

    # Scatter de páginas
    for ring in ["Core", "Focus", "Expansion", "Peripheral"]:
        mask = df["ring"] == ring
        fig.add_trace(go.Scatter(
            x=df[mask]["x"], y=df[mask]["y"],
            mode="markers",
            name=f"{ring} ({mask.sum()})",
            marker=dict(
                color=color_map[ring],
                size=df[mask]["weight"] * 20 + 5, # Tamaño = autoridad
                opacity=0.8,
```

```

    ),
    hovertemplate=(
        "<b>{%customdata[0]}</b><br>"
        "Anillo: " + ring + "<br>"
        "Distancia: {%customdata[1]:.4f}<br>"
        "Clicks: {%customdata[2]}<br>"
        "Peso: {%customdata[3]:.3f}<extra></extra>"
    ),
    customdata=df[mask][["url", "distance_to_centroid",
                        "clicks", "weight"]].values,
))

# Centroide
fig.add_trace(go.Scatter(
    x=[0], y=[0], mode="markers+text",
    marker=dict(symbol="star", size=16, color="black"),
    text=["Site Center"], textposition="bottom center",
    showlegend=False,
))

fig.update_layout(
    title=f"Mapa semántico - Focus: {site_metrics['focus_score']} | "
        f"Radius: {site_metrics['semantic_radius']} | "
        f"Drift: {site_metrics['drift_score']}",
    xaxis=dict(showgrid=False, zeroline=False, showticklabels=False),
    yaxis=dict(showgrid=False, zeroline=False, showticklabels=False,
                scaleanchor="x"),
    plot_bgcolor="white",
)
return fig

```

Tooltip unificado (ambos modos)

```

URL: /categoria/articulo-ejemplo
Anillo: Focus
Cluster HDBSCAN: 3 (tecnología)
Distancia al centroide: 0.34
PageRank normalizado: 0.71
Clicks (90d): 1.243
Peso combinado: 0.65
Canibaliza con: /blog/articulo-similar (sim: 0.94)

```

Análisis de brechas (Gap Analysis)

El usuario introduce un **topic objetivo** en texto libre. El sistema:

1. Genera el embedding `v_target` del topic.
2. Dibuja una flecha desde `c_site` hacia `v_target`.
3. Calcula el punto medio de esa trayectoria y busca las 5 URLs del corpus más cercanas a ese punto (contenido que podría reforzarse).
4. Identifica las coordenadas del espacio vacío en esa dirección (zona sin URLs → brecha de contenido confirmada).
5. Sugiere el título semántico aproximado del contenido que debería crearse para rellenar la brecha.

```
def gap_analysis(centroid: np.ndarray, target_text: str,
                 coords_2d: np.ndarray, df: pd.DataFrame,
                 model) -> dict:
    v_target = model.embed([target_text])[0]

    # Midpoint en espacio de alta dimensión
    midpoint_hd = (centroid + v_target) / 2

    # URLs más cercanas al midpoint
    distances = np.linalg.norm(
        np.vstack(df["vector"].values) - midpoint_hd, axis=1)
    df["gap_distance"] = distances
    closest = df.nsmallest(5, "gap_distance")[["url", "gap_distance"]]

    return {
        "target_vector": v_target,
        "closest_urls": closest,
        "gap_confirmed": bool((distances < distances.mean()).sum() < 3)
    }
```

8. Análisis de deriva semántica

Lista las URLs que más alejan el centroide del sitio de su foco semántico principal. Útil para decisiones de poda o consolidación de contenido.

```
def drift_analysis(df: pd.DataFrame, centroid: np.ndarray,
                  top_n: int = 10) -> pd.DataFrame:
    """
    Para cada URL, calcula cuánto se desplazaría el centroide
```

```
si esa URL fuese eliminada del corpus. Las URLs con mayor
delta de desplazamiento son las que más distorsionan el foco.
"""

results = []
vectors = np.vstack(df["vector"].values)
weights = df["weight"].values

for i, row in df.iterrows():
    mask = df.index != i
    c_without = np.average(vectors[mask], axis=0,
                           weights=weights[mask])
    delta = np.linalg.norm(c_without - centroid)
    results.append({"url": row["url"], "centroid_delta": delta})

return (pd.DataFrame(results)
        .sort_values("centroid_delta", ascending=False)
        .head(top_n))
```

9. Outputs exportables

fichero	contenido
scatter_umap.html	Scatter UMAP interactivo con clusters HDBSCAN (vista técnica)
ring_map.html	Mapa de anillos concéntricos Core/Focus/Expansion/Peripheral (vista cliente)
urls_report.csv	Todas las URLs con x, y, cluster, ring, distancia, weight, clics, PR
cannibalization.csv	Pares de canibalización con similitud y URL dominante
drift_top10.csv	URLs con mayor desviación del centroide
gap_analysis.json	Resultado del análisis de brechas por topic objetivo
site_summary.json	Focus Score, Semantic Radius, Drift Score y conteos por anillo

10. CLAUDE.md para Claude Code

```
# CLAUDE.md - Semantic Authority & Performance Mapper
```

Arquitectura del proyecto

```
sem-seo-engine/
├─ data/
│   ├── crawl.csv          # Exportación Screaming Frog (ver schema en README)
│   └─ gsc.csv             # Exportación GSC (ver schema en README)
├─ src/
│   ├── extraction.py      # extract_main_content(), playwright_extract()
│   ├── embeddings.py      # get_representative_chunk(), rate limiter
│   ├── graph.py           # build_link_graph(), pagerank
│   ├── engine.py          # SemanticEngine (clase principal)
│   ├── analysis.py        # drift_analysis(), gap_analysis(), detect_cannibaliza
│   └─ visualization.py    # build_plotly_map(), streamlit_app()
├─ outputs/                # Generado por el pipeline
└─ requirements.txt
```

Schema crawl.csv

```
url (string), status_code (int), indexable (bool),
inlinks (int), content_type (string), html (string)
```

Schema gsc.csv

```
url (string), clicks (int), impressions (int),
ctr (float), position (float)
```

Reglas de implementación

1. Normalización de URLs: lowercase + strip trailing slash + eliminar parámetros utm_*, fbclid, gclid antes de cualquier join.
2. Extracción de contenido: trafilatura primero. Si resultado < 150 tokens o None, fallback con Playwright. Si sigue siendo None, marcar como low_content y excluir del pipeline (no lanzar excepción).
3. Embeddings: usar rate limiter con exponential backoff. Procesar en batches de 100 chunks máximo por llamada. Modelo: text-embedding-3-large (OpenAI) o voyage-large-2-instruct.
4. Vectorización por página: NO usar max-pooling dimensional. Usar chunk representativo: embedding del chunk con mayor similitud coseno al centroide del documento.
5. UMAP: aplicar PCA a 50 dims antes de UMAP si corpus > 200 URLs. n_neighbors adaptativo según get_umap_config(n_urls). Siempre fijar random_state=42 para reproducibilidad.

6. Clustering: HDBSCAN post-UMAP. `min_cluster_size = max(5, n_urls // 50)`.
Puntos con `cluster == -1` se muestran en gris en el mapa.
7. Canibalización: calcular similitud coseno entre todos los pares ($O(n^2)$), optimizar con faiss si `n > 2.000`.
Umbral default: 0.92.
8. Logging: cada fase del pipeline debe loggear inicio, fin y conteo de URLs procesadas. Usar logging estándar de Python, nivel INFO.
9. Errores: nunca silenciar excepciones. Usar `try/except` solo para URLs individuales (aislar el fallo, continuar con el resto).
10. Tests: añadir test unitario para `normalize_url()`, `get_representative_chunk()` y `detect_cannibalization()`.
Usar `pytest` con `fixtures` en `tests/conftest.py`.

11. Limitaciones conocidas y próximos pasos

limitación	impacto	solución futura
Canibalización $O(n^2)$	Lento en corpus > 2.000 URLs	Sustituir por búsqueda ANN con FAISS
Embeddings por URL, no por intención	No distingue keyword cannibalization de topic cannibalization	Integrar GSC queries por URL para embedding de intención
UMAP no determinista entre ejecuciones	Coordenadas cambian entre runs	<code>random_state=42</code> mitiga; pendiente evaluación de estabilidad
Sin soporte multilingüe explícito	Sitios con varios idiomas generan clusters artificiales por idioma	Separar corpus por <code>hreflang</code> antes de vectorizar
Playwright requiere infraestructura adicional	Coste y complejidad operativa	Evaluar Crawl4AI como alternativa más ligera